Exam Management Portal

Datesheet, roll number slips and seating plans - without the manual work

Stack: React (frontend) + FastAPI (backend)
Inputs: Dummy student and course data (provided) + rooms and their capacities (provided)
Outputs: A clash-free datesheet, printable roll number slips, and a seating plan for every time slot

What this portal does - in one paragraph

The exam cell loads students, courses and rooms once. The user then defines the exam time slots. The system builds a datesheet that respects student clashes and room capacity, prints a roll number slip for every student for every paper, and generates a seating plan for each slot in which students of the same paper are never placed in two adjacent columns of the same room. Rooms are packed efficiently, so no hall is opened half-empty while another overflows.

In scope (this phase)

- Time slot definition by the user
- Automatic datesheet generation
- Roll number slip printing
- Seating plan per time slot and room
- Efficient room utilisation

Out of scope (deferred)

- Computer vision (CV) of any kind
- Chair detection from room photos
- Candidate picture analysis or auto-deletion
- Teacher / superintendent allotment
- Online exam conduct or result marking

Scope note: CV-based work from the original notes is deliberately excluded from this PRD, as decided on 5 November 2026. Nothing in the design should depend on it. It can be added later as a separate phase without changing the three core modules.

Field	Detail
Document	PRD - Exam Management Portal, v1.1
Audience	Development team (React + FastAPI), project supervisor, exam cell
Status	Ready for development. Two key rules were confirmed on 5 November 2026 (Section 10.4): one paper per student per day, and same paper allowed in the same column. Remaining open items in Section 10.4 do not block starting.
Data	Dummy / seed data only for development and testing. Real data replaces it through the same import format.

Contents and how the portal is used

Section	Title
1	Problem, goals and success measures
2	Users, roles and end-to-end workflow
3	Input data: students, courses, enrolments, rooms
4	Time slots and datesheet generation
5	Room utilisation rules
6	Roll number slips
7	Seating plan and the two-column rule
8	System architecture, data model and API
9	Screens, validation and non-functional requirements
10	Acceptance criteria, delivery plan and decisions

1. Problem, goals and success measures

Making a datesheet by hand means checking every student against every paper for clashes, then matching paper sizes against room capacities. Making a seating plan by hand repeats that work for every slot, and copying between neighbouring students is hard to prevent with a manual layout. Both jobs are slow, and a single missed clash is discovered only when students complain.

Goals

- **G1 - No clashes, ever.** A student is never scheduled for two papers in the same slot, has at most one paper on any one day, and no paper is scheduled twice.
- **G2 - Capacity is respected.** A slot is only scheduled when the rooms assigned to it can seat everyone appearing in it.
- **G3 - Fair seating.** In any room, students of the same paper do not sit in two adjacent columns (Section 7 gives the exact, testable rule).
- **G4 - Rooms are well used.** The system uses the fewest rooms that fit, and fills them evenly (targets in Section 5).
- **G5 - Print-ready output.** Datesheet, slips and seating plans print cleanly from the browser with no manual re-formatting.

Success measures (test data)

Measure	Target
Student clashes in a generated datesheet	0
Slots scheduled over room capacity	0
Two-column rule violations in a published seating plan	0
Average room utilisation per slot (Section 5)	85% or higher, where the data allows it
Time to generate datesheet + all seating plans for the dummy dataset	Under 10 seconds
Students with a slip for every enrolled paper	100%

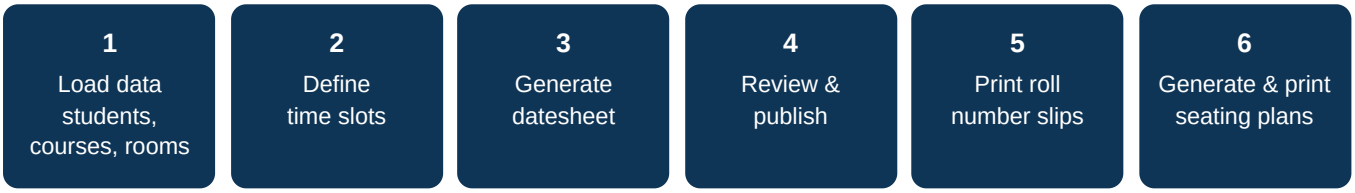
2. Users, roles and end-to-end workflow

2.1 Roles

Role	Can do	Cannot do
Exam Cell Admin (primary user)	Load and edit data, define slots, generate and publish datesheet, print slips, generate and print seating plans	- (full access in this phase)
Viewer (e.g. supervisor, department staff)	View the published datesheet and seating plans, print them	Edit data, regenerate or re-publish
Student	Receives a printed roll number slip only	No login in this phase

Single-admin use is acceptable for the first release, but login and the two roles above are required so that a published datesheet cannot be changed accidentally. Student logins and a student portal are out of scope.

2.2 End-to-end workflow



1. **Load data.** Admin uploads or confirms the seed data (Section 3). The system validates it and reports every error before anything else can run.
2. **Define time slots.** Admin creates the exam session (e.g. Final Term) and its slots: date, start time, end time. Slots can be added, edited or deleted until the datesheet is published.
3. **Generate datesheet.** One click runs the generator (Section 4). The result is a draft: every paper placed in a slot, or a clear list of papers that could not be placed and why.
4. **Review and publish.** Admin can regenerate, or drag a paper to a different slot (the system re-checks clashes instantly), then publishes. Publishing freezes that version.
5. **Print slips.** Slips are generated from the published datesheet only - never from a draft - so a printed slip can never contradict the final datesheet.
6. **Generate seating.** For each slot, the system allocates rooms and seats (Sections 5 and 7). Admin prints one seating chart per room per slot, plus an attendance list per room.

> **Version rule:** If a published datesheet is changed, the system must mark all slips and seating plans from the old version as superseded, and require them to be regenerated. Old and new prints must never be mixable: every print carries the datesheet version and generation date.

2.3 Core principles

- **Hard rules are never broken silently.** Clash, capacity and two-column rules are hard constraints. If they cannot all be met, the system reports it instead of producing a wrong plan.
- **Deterministic.** The same input data and slots always produce the same datesheet and seating plan, so a bug can be reproduced and a print can be re-issued identically.
- **Explainable.** Every unplaced paper and every relaxed preference is listed with a reason the admin can act on.

3. Input data: students, courses, enrolments, rooms

The portal works from data that is provided to it. For development and testing this is dummy (seed) data in exactly the format below, so the same import works unchanged when real data is supplied later. Data is loaded per exam session by CSV upload or the seed script, and can also be added and edited on screen.

3.1 Student

Field	Required	Notes
`student_id`	Yes	Unique, stable ID used internally
`roll_no`	Yes	Unique. Printed on the slip and used for seating order
`name`	Yes	Full name as it should appear on the slip
`program / batch / section`	Yes	e.g. BS CS, Batch 5, Section A. Used for grouping and filtering
`photo`	No	Optional in this phase. If absent, the slip prints without a photo box content

3.2 Course (paper) and enrolment

Field	Required	Notes
`course_code`	Yes	Unique, e.g. CS-301
`course_title`	Yes	Printed on datesheet and slip
`program / semester`	Yes	Which cohort the paper belongs to
Enrolment	Yes	A list linking `student_id` to `course_code`. A paper's strength = its enrolled student count. This is the single source for clashes, slips and seating.

3.3 Room

Field	Required	Notes
`room_id / name`	Yes	e.g. R-101. Unique
`building / floor`	No	Printed on slips and seating charts so students can find the room
`capacity`	Yes	Total usable seats for an exam. Must be a positive number
`rows x columns`	Yes	Seat grid layout, e.g. 6 rows x 5 columns = 30 seats. Rows x columns should equal capacity; if it does not, the smaller value is used and the admin is warned
`is_available`	Yes	Unavailable rooms are never allocated

3.4 Dummy data specification (for development)

- A seed dataset must ship with the project so any developer can run the full flow immediately: students across at least 3 batches / sections, courses for each batch, realistic enrolments, and rooms of different capacities.
- The seed must include the difficult cases, not only the easy ones: one paper larger than the largest single room, two papers that share many students, and one slot where total demand is close to total room capacity.
- Seed data is visibly dummy (fictional names, roll numbers in a reserved pattern) and is never presented as real institute data.

3.5 Data validation on load (FR-D1 to FR-D4)

- **FR-D1** Reject duplicate roll numbers, duplicate course codes and duplicate room names, listing each duplicate.
- **FR-D2** Reject enrolments that reference a missing student or course.
- **FR-D3** Warn when a paper has zero enrolled students or a room has zero / missing capacity.
- **FR-D4** Show a load summary - counts of students, courses, enrolments and rooms, and total seat capacity - before generation is enabled.

4. Time slots and datesheet generation

4.1 Time slots (FR-T1 to FR-T4)

- **FR-T1** The user defines slots inside an exam session: `date`, `start_time`, `end_time`, and an optional label (e.g. Morning / Evening).
- **FR-T2** Slots on the same date must not overlap in time. The system blocks saving an overlapping slot.
- **FR-T3 - One paper per student per day (confirmed, hard rule).** A student may appear in at most one paper on any date, across all slots that day. This is fixed - not a configurable default - and cannot be relaxed by the admin.
- **FR-T4** The system shows total room capacity per slot before generation.

4.2 Datesheet rules - hard constraints

ID	Rule	Meaning
H1	No student clash	Two papers that share even one enrolled student can never be placed in the same slot.
H2	One placement per paper	Every course is scheduled exactly once, in exactly one slot.
H3	Capacity per slot	Total students appearing in a slot must not exceed the total capacity of available rooms for that slot.
H4	One paper per student per day	No student is scheduled for more than one paper on the same date, in any slot. See FR-T3.

4.3 Datesheet preferences - soft constraints (in priority order)

1. **S1 - Balance the slots.** Spread students evenly across slots instead of filling the first slots and leaving the rest empty; balanced slots make seating and room use easier.
2. **S2 - Keep a cohort together in time.** Papers of the same batch / semester that share students must fall on different days (H4); otherwise, spread them across days where possible.
3. **S3 - Larger papers earlier.** Place the largest papers first while the most slot choice remains; this is also how the generator works (4.4).
4. **S4 - Respect admin pins.** A paper the admin has pinned to a slot stays there, provided no hard rule breaks. A pin that breaks a hard rule is refused with the reason.

4.4 Generation algorithm (required behaviour)

1. Build the clash graph: one node per paper; an edge between two papers if they share at least one student.
2. Sort papers by difficulty: highest enrolled strength first, then highest number of clashes.
3. For each paper, try slots in order of (a) no hard-rule break, (b) lowest current slot load. Place it in the first slot that satisfies all hard rules and best satisfies S1-S2.
4. If a paper fits nowhere, leave it unplaced - do not force it - and record the blocking reason (which rule, and which already-placed papers block each slot).
5. After placement, run a verification pass that re-checks H1-H4 from scratch on the finished datesheet. Generation only succeeds if verification passes.

> **Output states:** (a) **Complete** - every paper placed, verification passed; the draft can be published. (b) **Incomplete** - some papers unplaced, each with a reason and a suggested fix (add a slot on another day, or add a room). An incomplete datesheet cannot be published.

4.5 Datesheet screen and outputs (FR-T5 to FR-T7)

- **FR-T5** Grid view: dates and slots as columns, papers in their slots, with per-slot student totals and capacity usage.
- **FR-T6** Manual adjustment by drag-and-drop or a move control, with instant clash re-checking; an invalid move is blocked and the clash is named.
- **FR-T7** Print / export the datesheet (print view and PDF download) showing date, slot time, course code, title, batch and student count per paper.

5. Room utilisation: rooms must be well used

Good utilisation means two things at once: do not open more rooms than needed, and do not leave one room nearly empty while another is full. The rules below apply both when checking a slot's capacity for the datesheet and when allocating rooms for seating.

5.1 Definitions

Term	Definition
Slot demand	Total students appearing in that slot, across all its papers
Allocated capacity	Sum of capacities of the rooms allocated to that slot
Utilisation	Slot demand / allocated capacity x 100
Wasted seats	Allocated capacity - slot demand

5.2 Allocation rules (FR-R1 to FR-R5)

- **FR-R1 - Fewest rooms first.** Allocate the smallest set of available rooms whose combined capacity meets the slot demand. Prefer fewer, fuller rooms over more, emptier ones.
- **FR-R2 - Best fit.** From the available rooms, prefer the combination that leaves the fewest wasted seats (best-fit decreasing: consider larger rooms first, then fill the remainder with the smallest room that covers it).
- **FR-R3 - Target utilisation.** Aim for 85% or higher utilisation per slot. If the best possible allocation is below 70%, the system flags it on screen with the reason (e.g. room sizes do not match demand) - it still generates, because the exam must run.
- **FR-R4 - Even fill.** When students are distributed across the allocated rooms (Section 7), fill rooms proportionally to their capacity, so no room is left with only a handful of students while another is packed, except where the two-column rule requires a split.
- **FR-R5 - One use per slot.** A room is allocated to at most one seating plan per slot, and never beyond its capacity or its rows x columns grid.

5.3 Worked example

Slot demand is 82 students. Available rooms: R-101 (capacity 60), R-102 (capacity 30), R-103 (capacity 30), Hall A (capacity 100).

Option	Rooms	Capacity	Utilisation	Decision
A	Hall A alone	100	82%	Acceptable
B	R-101 + R-102	90	91%	**Chosen - best fit, fewest wasted seats (8)**
C	R-101 + R-102 + R-103	120	68%	Rejected - extra room, low use

Tie-break: if two options give the same wasted seats, choose the one with fewer rooms; if still tied, choose rooms in the same building / floor first, then by room name, so the result is deterministic.

5.4 What the admin sees

- Per slot: demand, rooms allocated, allocated capacity, utilisation % and wasted seats.
- Per room in the seating view: seats used / capacity and its utilisation %.
- A session summary: average utilisation across all slots, and any slot flagged below 70%.

> **Note on splitting papers:** A paper larger than any single room is split across rooms. A paper may also be split to satisfy the two-column rule (Section 7). Every split is shown on the seating plan and on the slips of the affected students - a student's slip always names the exact room they will sit in.

6. Roll number slips

Slips are generated from the **published** datesheet together with the seating allocation, so each slip can carry the student's room and seat. If seating has not been generated yet, slips may be printed without room / seat, and re-printed complete once seating exists; the slip must show which state it is in.

6.1 Slip content (FR-S1)

EXAMINATION ROLL NUMBER SLIP **Roll No: [roll_no]**

Exam Session: Final Term | Datesheet v1

Student: [name] **Program / Batch:** [program - batch - section]

Date	Time	Course	Room	Seat
[date]	[slot time]	[code] - [title]	[room]	[seat]

One row per enrolled paper, in date order. (Layout shown as a specimen - fields are the requirement, not this exact artwork.)

Student signature: _____ Invigilator signature: _____

- Header: institute / session name (configurable text), slip title, datesheet version.
- Student block: name, roll number, program, batch, section, and photo if provided.
- Paper table: one row per paper the student is enrolled in - date, slot time, course code and title, room and seat (after seating).
- Footer: signature lines and a short instructions line (configurable text, e.g. reporting time).

6.2 Slip rules (FR-S2 to FR-S6)

- **FR-S2 - One slip per student** covering all their papers (not one slip per paper), sorted by date and time.
- **FR-S3 - Print in bulk and singly:** all students, by batch / section, by course, or a single student by roll number (for re-issuing a lost slip).
- **FR-S4 - Print layout:** slips are laid out for A4 printing with clean page breaks between slips - a slip never splits across two pages. A compact layout (multiple slips per page) may be offered, with the same no-split rule.
- **FR-S5 - Completeness check before printing:** the system verifies every enrolled student appears exactly once and every paper row matches the published datesheet; any mismatch blocks printing and is listed.
- **FR-S6 - Re-print safety:** a re-print is identical to the original for the same datesheet version, and is marked "Duplicate" only if the admin chooses that option.

7. Seating plan and the two-column rule

A seating plan is generated **for every time slot**, and within a slot, **for every allocated room**. It assigns each appearing student to one exact seat, identified by room, row and column (e.g. R-102, Row 3, Column 2 - shown as R3-C2 on charts).

7.1 The rule, stated precisely

> **Two-column rule (hard constraint):** In any room, students of the same paper must never occupy two adjacent columns. If a student of Paper A sits in Column 1, no student of Paper A sits in Column 2 of that room in any row. Paper A may appear again in Column 3 or later, provided its neighbouring columns (2 and 4) hold a different paper - or are empty.

This is the testable form of the requirement "same 2 columns par same paper ka student na ho". It prevents the most common copying pattern - looking sideways at a neighbour doing the same paper - while still letting a large paper use several, separated columns of one room, and letting one column be filled front-to-back with the same paper (C1).

7.2 Companion rules (also hard, unless stated)

- **C1 - Front and back - same paper in the same column is allowed (confirmed).** Within one column, consecutive rows may hold the same paper - for example, a whole column assigned to Paper A, with A students in Rows 1, 2, 3 and 4. This is explicitly allowed and is not a violation. Only side-by-side (adjacent) columns must differ. Front-back separation is not required in this phase.
- **C2 - One student, one seat** per slot; one seat, at most one student. No seat is double-booked and no appearing student is left unseated.
- **C3 - Capacity and grid:** seats used in a room never exceed its rows x columns grid or its capacity.
- **C4 - Deterministic order:** within a paper, students are seated in roll number order, so an invigilator can find a student predictably and an attendance check is simple.
- **C5 (preference) - Mix papers in a room:** where a slot has two or more papers, prefer rooms containing at least two papers, because a single-paper room cannot satisfy the two-column rule without wasting every second column (see 7.4).

7.3 Example: one room, three papers, rule satisfied

Room grid 4 rows x 5 columns. Columns are assigned A, B, A, C, B - no two adjacent columns share a paper.

Seat	Col 1 - A	Col 2 - B	Col 3 - A	Col 4 - C	Col 5 - B
R1	A-01	B-01	A-05	C-01	B-05
R2	A-02	B-02	A-06	C-02	B-06
R3	A-03	B-03	A-07	C-03	-
R4	A-04	B-04	A-08	C-04	-

Reading it: Column 1 (A) and Column 2 (B) differ, Column 2 (B) and Column 3 (A) differ, and so on for every adjacent pair. A student of Paper A looking left or right always sees a different paper.

7.4 The single-paper case

If a room would contain only one paper, adjacent columns cannot differ. The system must, in this order: (1) try to move another paper of the same slot into the room, (2) try to move this paper's students into rooms that already hold other papers, (3) only if neither is possible, seat the paper using alternate columns and leave the columns between them empty - and flag the room on screen as "single-paper, alternate columns used", with its reduced effective capacity counted in the utilisation figures. It must never silently place the same paper in adjacent columns.

7.5 Seating generation algorithm (required behaviour)

1. **Collect the slot's demand:** every paper in the slot, with its enrolled students sorted by roll number.
2. **Allocate rooms** using the Section 5 rules (fewest rooms, best fit, target utilisation).
3. **Distribute papers to rooms:** split papers across rooms where needed so that, as far as possible, every room holds at least two papers and no paper is stranded alone (C5, 7.4). Larger papers are distributed first.
4. **Assign columns inside each room:** list the room's papers, then assign columns in a round-robin sequence of different papers (A, B, A, C, B ...) so no two adjacent columns repeat a paper. A paper with more students gets more columns, spaced apart.
5. **Fill seats:** fill each column top to bottom (Row 1 downwards) with that paper's students in roll number order.
6. **Verify:** re-check the finished plan from scratch - C2, C3, the two-column rule for every adjacent column pair in every room, and that every appearing student is seated exactly once. Publish only if verification passes.
7. **Report:** show per-room utilisation, any single-paper fallback (7.4), and any student who could not be seated, with the reason. An unseated student blocks publishing that slot's plan.

7.6 Seating outputs (FR-P1 to FR-P4)

- **FR-P1 - Room seating chart** (one per room per slot): grid of rows x columns, each cell showing roll number (and name in the detailed version), with the paper code per column clearly labelled. Print-ready for display outside / inside the room.
- **FR-P2 - Attendance list** (one per room per slot): students in seat order with roll number, name, paper and a blank signature column.
- **FR-P3 - Slot summary:** every room used in the slot, papers in it, students seated, utilisation %, and rule verification status.
- **FR-P4 - Student lookup:** enter a roll number and see that student's date, slot, room and seat for every paper - the same data that appears on the slip.

7.7 Regeneration and manual changes

- Regenerating seating for a slot replaces that slot's plan only; other slots are untouched.
- Manual seat moves are allowed before publishing the plan, but every move is re-checked live against the two-column rule and capacity; a breaking move is blocked with the reason.
- Once a slot's seating is published, regenerating it requires an explicit confirmation, and slips for that slot are marked superseded (Section 2.2 version rule).

> **Edge cases that must be handled:** a paper larger than all rooms combined in a slot (block, with the shortfall stated); a room whose grid does not match its capacity (use the smaller, warn); a student enrolled in two papers in the same slot (impossible if the datesheet is valid - if found, it is a datesheet bug and must be reported, not seated around); zero students in a slot (no plan needed, shown as empty).

8. System architecture, data model and API

8.1 Architecture

Layer	Technology and responsibility
Frontend	**React** single-page app: data screens, slot manager, datesheet grid, slip print views, seating charts. Print layouts are CSS print views of the same screens - no separate reporting tool.
Backend	**FastAPI** (Python): REST API, validation, and the three engines - datesheet generator, room allocator, seating generator. Generation runs server-side so results are identical for every user.
Database	Relational database (PostgreSQL preferred; SQLite acceptable for a local prototype with the same schema). All generated results are stored, versioned, and re-printable - never recomputed differently on print.
Files	CSV import for students, courses, enrolments and rooms. PDF / print output generated in the browser from stored data.

No CV, image-processing or machine-learning component is included in this phase. No external service is required; the portal must run fully on the institute network or a single machine for the prototype.

8.2 Data model (core tables)

Table	Key fields
`exam_session`	id, name, status (draft / published), version
`student`	id, roll_no (unique), name, program, batch, section
`course`	id, course_code (unique), title, program, semester
`enrolment`	student_id, course_id (unique pair)
`room`	id, name (unique), building, floor, capacity, rows, columns, is_available
`time_slot`	id, session_id, date, start_time, end_time, label
`datesheet_entry`	id, session_id, version, course_id, slot_id, is_pinned
`seat_assignment`	id, session_id, version, slot_id, room_id, student_id, course_id, row_no, col_no
`user`	id, name, login, role (admin / viewer)

8.3 API endpoints (minimum)

Endpoint	Purpose
`POST /api/import/{students|courses|enrolments|rooms}`	CSV load with validation report
`GET, POST, PUT, DELETE /api/students, /api/courses, /api/rooms`	On-screen data management
`GET, POST, PUT, DELETE /api/sessions/{id}/slots`	Time slot definition
`POST /api/sessions/{id}/datesheet/generate`	Run generator; returns draft + unplaced list with reasons
`PUT /api/datesheet/entries/{id}`	Move / pin a paper, with clash re-check
`POST /api/sessions/{id}/datesheet/publish`	Freeze a version
`GET /api/sessions/{id}/slips?batch=&course=&roll_no=`	Slip data for print, bulk or single
`POST /api/sessions/{id}/slots/{slot_id}/seating/generate`	Room allocation + seating for one slot
`GET /api/sessions/{id}/slots/{slot_id}/seating`	Charts, attendance lists, slot summary, student lookup

All list endpoints support the filters their screens show. All generate / publish endpoints return a structured report (placed, unplaced, violations, utilisation), not only a success flag.

9. Screens, validation and non-functional requirements

9.1 Screens (minimum set)

Screen	Must show / do
Dashboard	Session status, data counts, total capacity, next actions in workflow order
Data: Students / Courses / Rooms	Table with search and filters, add / edit / delete, CSV import with error report
Time Slots	Session settings (FR-T3), slot list by date, capacity per slot, overlap protection
Datesheet	Grid view (FR-T5), generate, drag / move with clash check, unplaced list, publish, print
Roll Number Slips	Filters (batch, course, roll number), completeness check, print preview
Seating Plans	Slot picker, per-room charts, attendance lists, utilisation and rule status, student lookup, print

9.2 General validation

- Every form validates before saving and names the exact field and problem.
- Deleting a student, course or room that is used in a published datesheet is blocked; in a draft, it warns and lists what will be affected.
- All generation actions are disabled, with the reason shown, until the data load summary (FR-D4) is clean of errors.

9.3 Non-functional requirements

Area	Requirement
Performance	Datesheet + seating generation for the dummy dataset completes in under 10 seconds; normal screens load in under 2 seconds on a local network
Print quality	Datesheet, slips and charts print on A4 without cut-off, overlap or split slips; tested in Chrome print preview
Reliability	Published versions are stored and re-printable identically at any later time
Usability	The full workflow (Section 2.2) is completable by the admin without developer help, in the order the dashboard presents it
Security (basic)	Login required; admin and viewer roles enforced on the API, not only hidden in the UI; passwords stored hashed
Browser	Latest Chrome / Edge. No mobile app in this phase; screens should remain usable on a laptop screen

10. Acceptance criteria, delivery plan and decisions

10.1 Acceptance criteria (all must pass on the dummy dataset)

- **AC1** Given loaded data and defined slots, when the admin generates the datesheet, then every paper is placed once, no student has two papers in one slot, no student has more than one paper on the same day, and no slot exceeds total room capacity - or each unplaced paper is listed with a specific reason.
- **AC2** Given a published datesheet, when slips are printed for a batch, then every student in that batch gets exactly one slip listing all their papers with correct dates and times.
- **AC3** Given a slot with at least two papers, when seating is generated, then a check of every room finds no two adjacent columns holding the same paper, no seat double-booked, and every appearing student seated exactly once. The same paper appearing in consecutive rows of the same column does not fail this check - it is allowed under C1.
- **AC4** Given the Section 5.3 example data, when rooms are allocated for 82 students, then the system chooses R-101 + R-102 (90 capacity, 91% utilisation).
- **AC5** Given a single-paper room is unavoidable, when seating is generated, then alternate columns are used, the room is flagged, and its reduced capacity is reflected in utilisation.
- **AC6** Given a published datesheet is changed, then earlier slips and seating plans are marked superseded and cannot be printed without a warning.

10.2 Delivery plan (suggested order)

Phase	Deliverable	Done when
M1	Project setup, data model, CSV import + validation, data screens, dummy seed data	Admin can load the seed data and sees a clean load summary (FR-D4)
M2	Time slots + datesheet generator with verification, grid screen, publish	AC1 passes
M3	Room allocation + seating generator with two-column verification, charts and attendance lists	AC3, AC4 and AC5 pass
M4	Roll number slips (bulk, filtered, single), student lookup, print polish	AC2 and AC6 pass; print check on A4 passes
M5	Roles and login, hardening, supervisor demo on dummy data	Full workflow (Section 2.2) demoed end to end without developer help

10.3 Risks and mitigations

Risk	Mitigation in this PRD
Not enough slots or rooms for the papers given	Generator reports exactly what is unplaced and why, with suggested fixes, instead of failing silently (4.4)
Room sizes do not match paper sizes, wasting seats	Best-fit allocation, paper splitting, and utilisation flags make waste visible and minimal (Section 5)
A slot contains only one large paper	Defined fallback: alternate columns, flagged, with honest utilisation (7.4)
Prints from an old datesheet version circulate	Version printed on every output; superseded marking (2.2, AC6)
Dummy data is too easy and hides real problems	Seed data must include the difficult cases listed in 3.4

10.4 Decisions - confirmed and still open

1. **Per-day limit - CONFIRMED (5 November 2026):** one paper per student per day. This is a hard rule (FR-T3, H4), not a default. A student must not be given two papers on the same date, even in different slots.
2. **Front-back / same-column rule - CONFIRMED (5 November 2026):** the same paper in the same column is allowed, including consecutive rows in that column (C1). Only adjacent, side-by-side columns must hold different papers.
3. **Slip layout:** one slip per page, or compact multiple slips per A4 page, as the default print layout.
4. **Room grid source:** confirm each room's rows x columns will be provided with the room data; if not, the admin must enter them before seating can run.
5. **Institute header and instructions text** for slips and charts: name, session naming, and the instruction line to print.

> **Definition of done for this phase:** On the dummy dataset, an admin can - without developer help - load data, define time slots, generate and publish a clash-free datesheet, print correct roll number slips for every student, and generate and print a seating plan for every slot that passes the two-column rule and meets the room utilisation rules in this document.

End of PRD v1.1 - Exam Management Portal (React + FastAPI). CV-based features, teacher / superintendent allotment and student logins are deferred to a later phase and are not part of this document's acceptance criteria.